

실시간 모의 선물 거래소

Binance 실시간 데이터 파이프라인

Binance USDS-M Futures의 실시간 호가창(order book)을 받아, 서버를 거쳐 모든 브라우저에 지연 없이 fan-out하는 reactive 파이프라인

Backend	Java 21 · Spring Boot 4 (WebFlux) · Reactor Netty · Jackson 3
Auth/DB	Spring Security · PostgreSQL (R2DBC)
Frontend	React + TypeScript (Vite) · TradingView Lightweight Charts
External	Binance USDS-M Futures — WebSocket (depth) · REST (kline)

실행 구조

```
OS
├─ java.exe 프로세스 1개
│   └─ JVM
│       └─ Spring Boot WebFlux 애플리케이션
│           ├── Spring Boot 부팅/설정/Bean 관리
│           ├── WebFlux 라우팅/컨트롤러/Mono-Flux 응답 처리
│           ├── Reactor Netty embedded 서버
│           ├── Project Reactor Flux/Mono 실행
│           └─ 내가 작성한 Controller/Service/Repository 코드
```

이 문서의 범위

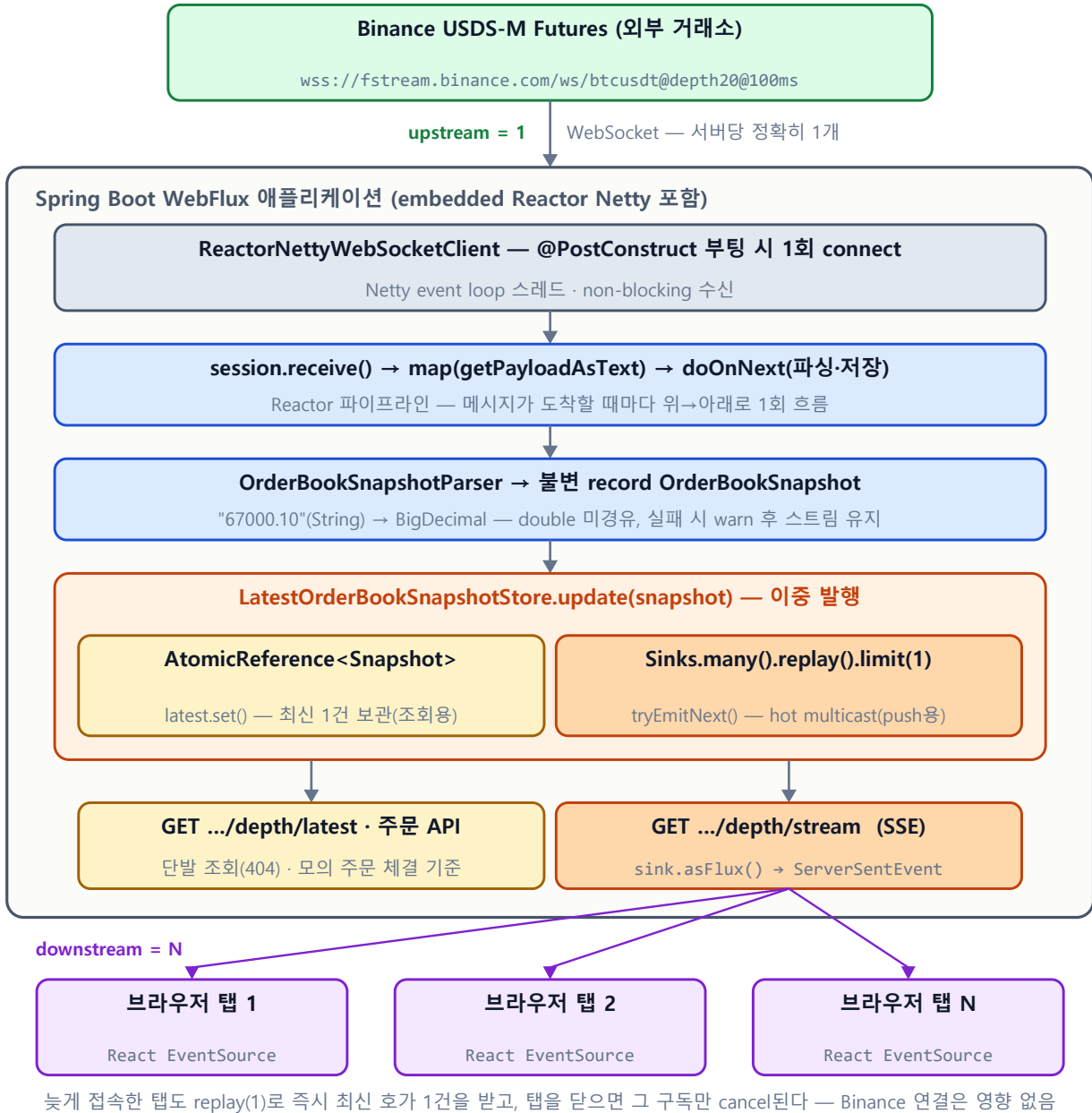
전체 프로젝트는 호가창 → 차트 → 회원가입/로그인 → 호가창 기준 모의 주문 → 계좌·포지션·PnL로 이어지는 학습용 모의 선물 거래소다. 이 문서는 그 중 모든 기능의 기준 가격 데이터를 만들어내는 **Binance 실시간 데이터 파이프라인** 파트만 다룬다.

왜 이 파트가 프로젝트의 핵심인가

- 호가창 표시, 캔들차트의 진행 봉, 모의 주문 체결가, 미실현 PnL까지 — 전부 이 파이프라인이 만드는 최신 호가 snapshot 하나를 기준값으로 동작한다.
- 외부 실시간 시스템(거래소 WebSocket) 연동, reactive 비동기 모델(cold/hot publisher), 락 없는 동시성 처리, 생명주기 분리 설계가 이 파트에 응축되어 있다.
- 직접 측정으로 원인을 좁힌 트러블슈팅 2건(중복 연결, 지역 차단)이 이 파트에서 나왔다 — 4페이지 참조.

GitHub: github.com/lee-gimoon · Sink Flow 인터랙티브 시각화: ([호스팅 링크](#) — [배포 후 기입](#)) · 2026.06

아키텍처 — 호가 데이터가 흐르는 길



런타임 관점

이 서버는 단일 JVM 프로세스에서 실행되는 Spring Boot WebFlux 애플리케이션이다. Spring Boot는 부팅·설정·Bean 관리를 맡고, WebFlux는 요청 라우팅과 **Mono/Flux** 응답 처리를 담당한다. 실제 HTTP/SSE/WebSocket 네트워크 I/O는 이 애플리케이션에 내장된 Reactor Netty 서버가 같은 JVM 프로세스 안에서 처리한다.

- 부팅 시 1회 연결** @PostConstruct가 Binance WebSocket을 연결한다. 이후 브라우저가 몇 명 붙든 서버↔Binance 연결은 1개로 유지된다.
- non-blocking 수신** Netty event loop가 100ms마다 도착하는 depth20 프레임(상위 20개 bid/ask)을 non-blocking으로 수신한다.
- 파싱** raw JSON을 불변 record OrderBookSnapshot으로 변환한다. 가격·수량은 String → BigDecimal 직행.
- 이중 발행** Store가 AtomicReference 교체(조회용)와 sink.tryEmitNext(push용)를 동시에 수행한다.
- SSE push** /depth/stream 구독자 전원에게 즉시 push. 새로 접속한 브라우저는 replay(1)로 최신 1건을 바로 받아 빈 화면 없이 시작한다.
- pull 조회** /depth/latest(디버깅용 단발 조회)뿐 아니라 모의 주문 API(POST /api/paper/orders)도 체결 직전에 AtomicReference에서 최신 snapshot을 꺼내 체결 기준 호가로 쓴다.

핵심 설계 결정

1. Binance 연결은 사용자가 아니라 서버가 소유한다

연결을 HTTP 요청이 아닌 **서버 생명주기**(@PostConstruct)에 묶었다. 1개여야 하는 upstream(서버↔Binance)과 N개여도 되는 downstream(브라우저↔서버)을 다른 트리거로 분리한 것. 브라우저가 전부 나가도 수신은 계속되고, 100명이 접속해도 Binance 연결은 1개다.

2. 폴링을 버리고 Hot Sink push로 전환

초기 구현은 `Flux.interval(100ms)`로 store를 폴링하고 `distinctUntilChanged`로 중복을 걸렀다. 이를 `Sinks.many().replay().limit(1)` 기반 push로 교체 — snapshot이 도착하는 순간 모든 구독자에게 전달되고, 빈 tick 낭비와 폴링 주기만큼의 지연이 사라졌다. 늦게 붙은 구독자도 `replay(1)`로 최신 1건을 즉시 받는다.

3. 락 없는 동시성 — AtomicReference + record snapshot

Writer는 WebSocket event loop 1개, Reader는 HTTP 요청 N개다.

여기서 동시성 처리의 목표는 Reader가 A/B가 섞인 깨진 snapshot을 보거나, 갱신된 참조를 보지 못하거나, snapshot의 정합성이 깨지는 일을 막는 것이다.

snapshot은 한 시점의 호가 값 묶음이라 record로 짧게 표현했다.

기존 snapshot을 고치지 않고 새 snapshot을 완성한 뒤, AtomicReference가 최신 참조만 한 번에 교체한다.

그 결과 Reader는 synchronized 없이도 이전 snapshot 또는 새 snapshot 중 하나의 완성본만 읽는다.

역할도 분리했다. AtomicReference는 pull용(단발 조회 + 주문 체결 기준), Sink는 실시간 push용이다.

4. 가격은 String → BigDecimal 직행, double 금지

Binance가 가격을 "67000.10" 문자열로 보내는 이유는 이진 부동소수점 오차 때문이다. `asDouble()`을 거치면 그 시점에 이미 값이 망가지므로, `asString() → new BigDecimal(String)` 경로만 사용한다. 거래소에서 1원 오차는 누군가의 손익이다.

5. 메시지 단위 장애 격리

파싱 예외를 메시지 1건 단위로 잡아 warn 로그만 남기고 스트림은 계속 흐르게 했다. 깨진 메시지 하나가 실시간 파이프라인 전체를 죽이는 일을 막는다.

6. WebSocket 중계 대신 SSE, Tomcat 대신 WebFlux

서버→브라우저는 단방향 push라 SSE로 충분하다(EventSource 자동 재연결 포함). blocking MVC로 SSE를 구현하면 연결 100개에 worker thread 100개가 묶이지만, WebFlux/Netty event loop는 쓸 데이터가 생긴 순간에만 잠깐 write를 처리하므로 적은 스레드로 많은 SSE 연결을 유지한다.

MVC + Tomcat이었다면? — 스레드 모델 비교

같은 SSE를 **thread-per-request(블로킹)** 모델로 구현하면 열린 연결 1개당 스레드 1개가 묶인다 — 대부분 다음 snapshot을 기다리며 노는데도. 이 앱은 100ms마다 갱신되는 호가를 다수 브라우저로 fan-out하는, 오래 열린 연결이 많은 워크로드라 연결과 스레드를 분리하는 event loop 모델이 맞았다.

관점	MVC + Tomcat (thread-per-request)	WebFlux + Netty (event loop)
SSE 연결 N개	연결마다 worker thread 점유 — 기본 풀(~200)에서 한계	적은 event loop 스레드가 다수 연결을 담당
이 프로젝트	오래 열린 idle 호가 연결이 스레드를 잡아둠	snapshot 도착 시에만 잠깐 write
DB 접근	JDBC(blocking) — 단순	R2DBC 필요 · blocking 호출 금지
난이도	스택트레이스 명확 · 디버깅 쉬움	연산자 체인 추적 어려움 · 학습곡선
적합한 곳	일반 CRUD · 트랜잭션	스트리밍 · 다수 long-lived 연결

* idle(유휴) = 연결은 열려 있지만 데이터가 안 흐르는 대기 시간. 호가 SSE는 100ms마다 잠깐 write하고 나머지 시간은 대부분 idle이다.

구체적으로 — 브라우저 300명이 동시에 붙으면?

MVC + Tomcat (블로킹)	WebFlux + Netty (event loop)
연결 300개 = 스레드 300개 점유 (대부분 대기)	적은 스레드(코어 수)가 300 연결을 모두 담당
기본 스레드 풀 ~200 → 201번째부터 막힘	연결이 늘어도 스레드는 거의 안 늘어남
스레드당 스택 ~1MB → 약 300MB 점유	새 snapshot 올 때만 잠깐 write

양쪽 모두 Binance 업스트림 연결은 **1개**로 같다. 차이는 다운스트림(브라우저)을 처리하는 방식 — 핵심은 **연결과 스레드를 분리**해 적은 스레드로 많은 long-lived 연결을 유지하는 것이다.

단, 위 표는 **블로킹** 모델 기준이다. Spring MVC도 SseEmitter(async)를 쓰면 idle 연결에 스레드를 묶지 않는다 — 그러면 'idle 연결 스레드 절약'은 WebFlux만의 이점이 아니다. 그럼 왜 WebFlux인가?

Netty란 무엇인가 — HTTP 서버가 아니라 네트워크 I/O 프레임워크

Netty는 **Java NIO 기반의 비동기 이벤트 기반 네트워크 프레임워크**다. TCP 소켓, event loop, non-blocking I/O, ByteBuf, ChannelPipeline/Handler 같은 저수준 네트워크 요소를 직접 다루기 쉽게 해준다.

따라서 Netty는 HTTP 전용 서버가 아니라, 여러 종류의 네트워크 서버와 클라이언트를 만들 수 있는 **네트워크 I/O 기반**이다.

HTTP 서버	Reactor Netty HTTP Server가 이 범주에 속한다. WebFlux의 기본 내장 서버다.
HTTP/WebSocket 클라이언트	WebClient, ReactorNettyWebSocketClient 같은 클라이언트 I/O의 기반이 된다.
TCP 서버/클라이언트	특정 프로토콜에 묶이지 않고 raw TCP 기반 프로그램을 만들 수 있다.
DB/캐시 프로토콜 클라이언트	PostgreSQL, Redis 같은 TCP 프로토콜 드라이버가 Netty 모델을 활용할 수 있다.
커스텀 프로토콜	직접 정의한 바이너리/텍스트 프로토콜도 ChannelPipeline으로 처리할 수 있다.

이 프로젝트에서 **Reactor Netty HTTP Server**는 Netty 위에 HTTP/SSE/WebSocket 서버를 만든 것이고, **r2dbc-postgresql**은 Netty 모델로 PostgreSQL 클라이언트 TCP I/O를 처리한다. 즉 HTTP 서버 엔진이 DB를 대신 처리하는 게 아니라, HTTP 서버와 DB 드라이버가 각각 Netty의 event loop/Channel 모델을 활용한다.

왜 WebFlux + Netty인가 — 스레드 효율과 스트림 적합성

이 앱에서 WebFlux + Netty를 고른 1차 이유는 **적은 event loop 스레드로 많은 long-lived 연결을 다루기 좋은 실행 모델**이기 때문이다. Binance WebSocket upstream은 계속 열려 있고, 브라우저 SSE 연결도 오래 유지되지만 실제 read/write는 snapshot이 오거나 전송할 때만 짧게 발생한다.

다만 이 장점은 **idle SSE 연결의 스레드 절약**만으로 설명하면 부족하다. MVC도 SseEmitter(async)를 쓰면 idle 연결에 스레드를 계속 묶지 않는다. 더 중요한 차이는 **Binance 수신 -> fan-out -> SSE 응답 -> DB 접근**까지 논블로킹 흐름을 유지할 수 있느냐다. MVC + JDBC는 DB 응답 동안 요청 처리 풀 스레드가 붙잡히고, WebFlux라도 JDBC 호출이나 block() 같은 blocking 호출을 event loop에서 직접 실행하면 event loop가 막힌다. 이 앱은 WebFlux + R2DBC로 DB까지 논블로킹 경로를 유지한다.

거기에 이 프로젝트가 특히 reactive와 맞는 추가 이유:

- **업스트림도 다운스트림도 Flux** — Binance WebSocket 수신(Flux<WebSocketMessage>)부터 브라우저 SSE 송신까지 같은 reactive 파이프라인으로 이어진다. MVC + SseEmitter로도 구현은 가능하지만, WebSocket 콜백/세션 기반 수신을 Sink나 emitter 관리 코드로 직접 이어줘야 한다.
- **Sinks 멀티캐스트 + replay(1)이 기본 제공** — '1 upstream → N downstream, 늦게 온 구독자에게 최신 1건 즉시'가 Sinks.many().replay().limit(1)와 sink.asFlux()로 끝난다. MVC + SseEmitter면 List<SseEmitter>를 직접 add/remove/순회 send하고 replay까지 수동 구현해야 한다.
- **백프레시·구독 취소를 다루는 틀이 있다** — 느린 클라이언트 대응과 탭 닫힘(cancel) 정리를 Flux 구독/취소 신호 안에서 일관되게 처리할 수 있다.

정직한 결론 — 이 앱 규모면 MVC + SseEmitter로도 가능하다. 그럼에도 WebFlux를 고른 건 스레드 효율을 **경로 끝까지(논블로킹 일관)** 유지하기 쉽고, '양 끝이 스트림'인 구조를 자연스럽게 표현하기 때문이다 — 문제 모양에 맞는 도구다. 평범한 CRUD·트랜잭션 위주였다면 MVC + Tomcat이 더 단순하고 옳았을 것이다.

왜 경로 전체가 논블로킹이어야 하는가 — event loop를 기다리게 만들면 안 된다

WebFlux + Netty의 핵심은 단순히 “스레드가 적다”가 아니다. 핵심은 **많은 HTTP/SSE/WebSocket 연결은 OS TCP socket과 Netty Channel로 유지하고, 적은 event loop 스레드는 read/write 이벤트가 생긴 연결만 짧게 처리하는 모델**이라는 점이다. 그래서 이 모델에서는 그 적은 스레드를 오래 기다리게 만들면 안 된다.

중간에 JPA/JDBC 같은 blocking 호출이 event loop 위에서 실행되면, DB 응답이 올 때까지 해당 event loop가 멈춘다. 그러면 그 요청 하나만 늦어지는 것이 아니라, 같은 event loop에 묶인 다른 socket 이벤트들도 같이 밀린다. 실제 서비스에서는 응답 지연 증가, SSE/WebSocket 전송 지연, 메시지 backlog, 메모리 압박, 타임아웃, 전체 처리량 저하로 이어질 수 있다.

따라서 경로 전체를 논블로킹으로 유지해야 하는 이유는 reactive 순수성을 지키기 위해서가 아니라, **중간의 blocking I/O 하나가 event loop를 기다리게 만들어 전체 스트리밍 처리량과 지연시간에 영향을 주기 때문**이다. Binance WebSocket 수신 → 파싱 → fan-out → SSE 응답 → DB 접근 중 한 구간이라도 DB 응답, 외부 API 응답, 파일 I/O 같은 대기 작업으로 event loop를 붙잡으면, 같은 event loop가 맡은 다른 연결들의 read/write 이벤트도 함께 밀릴 수 있다.

그래서 이 프로젝트에서 R2DBC를 쓰는 이유도 단순히 타입을 Mono/Flux로 맞추기 위해서가 아니다. DB I/O를 실제로 논블로킹으로 처리하는 주체는 R2DBC 드라이버이고, Mono/Flux는 그 결과를 onNext/onError/onComplete 신호로 표현해 파싱, fan-out, SSE 응답, DB 접근 흐름을 하나의 reactive pipeline으로 조합하는 역할을 한다.

같은 event loop에 묶인다는 뜻

기준	TCP 연결 1개 = Netty Channel 1개, EventLoop 1개 = 스레드 1개
묶인다는 뜻	같은 EventLoop에 등록된 여러 Channel의 read/write/close 이벤트가 같은 스레드에서 처리된다.
예시	reactor-http-nio-1: 브라우저 A SSE, 브라우저 C SSE, 브라우저 F HTTP, Binance WebSocket
blocking 결과	이 스레드가 300ms DB 응답을 기다리면 A/C write, F 요청, Binance read도 함께 지연될 수 있다.

가정: DB 저장이 추가된다면 — 기다림과 스레드 점유는 다르다

현재 Binance 실시간 데이터 파이프라인은 WebSocket으로 snapshot을 수신하고, 파싱한 뒤 Store와 Sink로 fan-out 한다. 이 경로에는 현재 DB 저장 로직이 없다. 다만 나중에 snapshot 이력 저장, 체결 로그 저장, 감사 로그 저장 같은 DB I/O가 이 파이프라인 중간에 추가될 수 있다.

이 문서에서 콜스택은 현재 스레드가 실행 중인 메서드 호출의 층을 뜻한다. Binance read → 파싱 → 저장 요청처럼 호출이 이어지는 동안 콜스택이 쌓이고, 메서드들이 return되면 콜스택이 비워져 event loop가 다음 Channel 이벤트를 처리할 수 있다.

이때 JPA/JDBC를 직접 호출하면 DB 응답을 기다리는 동안 현재 event loop 콜스택이 blocking된다. 반면 R2DBC라면 DB 요청을 걸고 현재 콜스택은 return된다.

가정: snapshot 저장 로직이 추가된 경우

Binance Channel read 콜스택

- snapshot 생성
- R2DBC save 요청 시작
- DB Channel에 SQL write 요청
- 현재 콜스택 return

이후 PostgreSQL 응답이 도착하면 DB Channel 쪽에서 별도의 read 이벤트가 발생한다.

DB Channel read 이벤트

- PostgreSQL 응답 수신
- Mono 완료/실패 신호
- DB 저장 결과를 필요로 하는 후속 reactive 단계 실행

여기서 중요한 차이는 “기다린다”와 “스레드를 붙잡고 기다린다”는 다르다는 점이다. DB 저장 결과가 필요한 다음 단계가 있다면, 그 단계는 DB 응답 이후에 실행되는 것이 맞다. 즉 R2DBC도 논리적 순서는 기다린다. 하지만 그 기다림이 현재 Java 콜스택과 OS 스레드 안에서 일어나지 않는다. R2DBC는 DB 요청을 보낸 뒤 “응답이 오면 이 후속 단계를 실행하라”는 상태를 reactive 구독/DB 요청 상태로 등록하고 현재 콜스택을 return한다. 그래서 event loop 스레드는 DB 응답을 기다리며 멈춰 있지 않고, 다른 Channel 이벤트를 계속 처리할 수 있다.

반면 JPA/JDBC는 DB 응답이 올 때까지 현재 호출 스레드가 socket read 같은 blocking 지점에서 멈춘다. 이 경우 기다림이 현재 콜스택 안에 남아 있기 때문에, 그 스레드가 Netty event loop라면 같은 event loop에 등록된 다른 SSE write, HTTP 요청, WebSocket read 이벤트까지 함께 밀릴 수 있다.

즉 R2DBC에서는 Binance Channel read와 DB Channel read가 하나의 계속 유지되는 콜스택이 아니다. 처음에는 Binance read 콜스택에서 DB 요청을 “걸고” 빠져나오고, DB 응답은 나중에 DB Channel 이벤트로 별도 콜스택에서 이어진다. 그래서 DB 응답 대기 시간이 Binance read 콜스택과 event loop를 붙잡지 않는다.

단, 실시간 fan-out을 DB 저장 완료 뒤에 할지, 먼저 fan-out하고 저장을 별도 비동기 side-effect로 분리할지는 설계 선택이다.

정합성 우선	snapshot 생성 → DB 저장 완료 → Store/Sink fan-out
실시간성 우선	snapshot 생성 → Store/Sink fan-out → DB 저장은 별도 비동기 처리

현재 호가 화면처럼 실시간성이 중요한 데이터라면 보통 fan-out을 먼저 하고, 이력 저장은 별도 비동기 흐름으로 분리하는 편이 자연스럽다. 어느 쪽이든 R2DBC의 핵심은 **DB 응답을 기다리는 동안 event loop 스레드를 붙잡지 않는**다는 점이다.

트러블슈팅

1. 같은 Binance 스트림이 호출 수만큼 중복 연결되던 문제

증상	초기엔 프론트의 '스트림 시작' 버튼이 POST /raw/start로 connect()를 호출하는 구조였다. 버튼 재클릭·새로고침·새 탭마다 Binance WebSocket이 1개씩 늘어났고, 재연결이 아니라 같은 시간대에 N개가 나란히 살아서 동일 데이터의 수신 트래픽·로그·CPU가 N배가 됐다.
원인	webSocketClient.execute(...)가 반환하는 Mono는 cold publisher — subscribe할 때마다 새 연결을 만든다. 근본 원인은 1개여야 하는 upstream 연결의 생성을 N번 발생하는 downstream 트리거(사용자 행동)에 묶은 설계였다.
해결	connect()를 HTTP 엔드포인트에서 떼어내 @PostConstruct로 이동 — 빈 생성 직후 정확히 1회만 subscribe된다. 연결 수가 사용자 행동과 완전히 분리됐다.
배운 점	cold/hot publisher 구분이 '이 코드가 몇 번 실행되는가'를 결정한다는 것. 그리고 외부 연결의 소유권은 요청이 아니라 서버 생명주기에 뒤야 한다는 원칙.

2. WebSocket이 연결은 되는데 메시지가 0건 — 지역 차단 진단

증상	캔들차트용 @kline 스트림을 구독하면 에러 없이 연결되지만 메시지가 한 건도 오지 않았다. 예외도 로그도 없어서 코드만 보서는 원인을 알 수 없는 상태.
진단	스트림을 계열별로 분리해 수신 메시지 수를 직접 측정했다. 결과 — 체결 계열(@kline:@aggTrade:@markPrice)은 전부 0건 , 호가 계열(@depth:@bookTicker)과 REST API는 정상 . 동일한 코드·연결 방식에서 스트림 계열별로만 결과가 갈리므로, 코드 문제가 아니라 네트워크/지역 차단으로 원인을 분리했다.
해결	과거 봉은 Binance kline REST로 backfill(공개 시세라 API 키가 불필요 → 브라우저가 직접 호출), 진행 봉은 이미 백엔드가 받고 있는 호가 SSE의 best ask로 OHLC를 갱신. 추가 연결 0개로 차트를 완성했다.
배운 점	'안 된다'를 '정확히 무엇이 어디까지 되는가'로 좁히는 측정 기반 디버깅. 진행 봉이 체결가가 아닌 호가 기준이라는 한계는 숨기지 않고 문서에 명시했다.

해결 경로 — 막힌 @kline을 빼고 차트를 두 경로로 완성

과거 봉 직접	브라우저 — HTTP GET —> Binance kline REST fapi.binance.com
호가창 · 진행 봉 서버 경유	브라우저 — SSE —> Spring 서버 — WebSocket —> Binance @depth fstream.binance.com

과거 봉은 브라우저가 Binance에 직접 요청(서버 안 거침), 진행 봉은 호가창이 쓰는 그 SSE의 **best ask**를 재활용한다 — 차트용 추가 연결 0개, 막힌 @kline WebSocket은 전혀 쓰지 않는다.

현재 한계와 다음 단계

정직하게 적는 현재 한계

- **partial depth20 snapshot** — 정밀한 로컬 오더북이 아니다. Binance가 100ms마다 보내는 상위 20레벨을 통째로 교체할 뿐, sequence 검증이 없다. 호가창 표시-간단한 모의 체결에는 충분하지만 정밀 체결 시뮬레이션에는 부족하다.
- **단일 서버 local fan-out** — 구독자 수백 명까지는 무난할 것으로 보지만, 1,000명 전후부터는 부하 테스트가 필요하다.
- **재연결-운영성 미구현** — 연결 끊김 시 지수 백오프 재연결, stale 호가 감지, health 표시는 운영 안정화 단계에서 보강할 항목이다.

다음 단계 (로드맵 기준)

- **정밀 로컬 오더북** — REST snapshot + diff depth 스트림을 U/u/ps 필드로 sequence 검증하며 유지하고, 불일치 시 snapshot 재동기화. 현재의 SSE/화면은 그대로 두고 내부 source만 교체한다.
- **운영성 보강** — health 엔드포인트, 연결 status(UP/DOWN/STALE), 지수 백오프 재연결, 설정 외부화.
- **확장 설계** — 선직렬화(구독자 N명에게 같은 JSON을 N번 만들지 않기), throttle/diff 전송, 다중 인스턴스 시 브로커 도입 검토.

이 파트가 받치는 다음 기능들

이 파이프라인이 만드는 최신 호가 snapshot은 이후 단계의 기준값이 된다 — 모의 주문은 best bid/ask부터 호가 레벨을 순서대로 소진하며 체결되고(주문:체결 = 1:N), 미실현 PnL은 mid price로 계산된다. 즉 이 문서의 파이프라인이 거래소 전체의 '가격 진실 공급원(source of truth)'이다.

본 문서는 프로젝트 진행에 따라 업데이트됩니다.